Structurally Valid, Semantically Wrong: Measuring the Unenforced-Rule Gap at a Web Application's Input Boundary

Zahid Rahimoon and Fida Hussain Khoso

Abstract—A web application's schema validation checks that an input is well formed. It does not check the business rules that decide whether the input is appropriate, and in the application studied here most of those rules are not enforced in code. This article measures how often inputs that pass validation break such rules, and whether inputs written by large language models (LLMs) differ from inputs produced in other ways. A Next.js testbed with an enumerated reference of 43 business rules received 3881 inputs through its real HTTP validation path, from seven generation conditions covering LLM, random, adversarial, boundary and reference inputs. Semantic validity was decided by deterministic rule functions for objectively decidable rules and by an independent judge model for the rest. Three results do not depend on the judge. Rules the application could enforce in code were broken by 7.9 % of structurally valid LLM inputs, and by 1.8 % once a date target driven by a prompt artefact is removed, below the pre-registered bound. Random structurally valid inputs broke rules more often than LLM inputs (54.5 % against 37.6 %), so the gap belongs to the unenforced boundary rather than to AI writers. And a rule layer costs at most 14 ms of p97.5 latency against seconds per request for an LLM judge, which saturates at eight concurrent clients. Results resting on judgement rules are exploratory: calibrated against designed conditions the judge caught 78.6 % of planted violations but flagged 14.1 % of valid inputs. No candidate layer reached the pre-registered target of 70 % recall at a 5 % false-positive rate. The study covers one application and CPU-only open models.

Keywords: input validation, business logic, large language models, web applications, software testing, empirical study

I. Introduction

A web application accepts an input only after it passes validation. In the JavaScript and TypeScript ecosystem that validation is usually a schema: a declarative object stating the type, presence, length, pattern, numeric range, enumeration and format of every field, checked on the client, again on the server, and finally by database constraints. Schemas of this kind are well understood [1], [2] and have been studied empirically at scale [3]. Other rules depend on what the application is for: a promotional code has to apply to the item being bought, a workshop cannot start in the past, a review is supposed to be about the workshop it is attached to. Some can be written as custom refinements, though in practice most are never written down. The distinction that matters is between the business rules an application enforces and the many it leaves unenforced.

That limitation is not new: Felmetsger et al. separated input-validation flaws, which admit a general specification, from logic vulnerabilities, which do not [4], and later work detected such flaws from behavioural traces, validation mismatches and over-permissive payloads [5]–[7]. What has changed is who writes the input. Large language models (LLMs) now fill forms, operate web applications on a user's behalf, and generate test data [8]–[10].

Recent work shows that LLM output which satisfies a schema is often wrong in meaning. Schema validity runs well ahead of semantic success for ordering agents, constrained decoders, tool interfaces and enterprise workflow generators [11]–[15]. All of this concerns model outputs that a tool, an API or a renderer consumes, and there a wrong value is discarded. At the input boundary of a web application the accepted value is stored and later shown to other users or to an assistant, and two questions remain open there. How often do LLM-written inputs that pass validation break the application's business rules? And is that rate different from the rate for other producers of structurally valid input? Web testing studies cannot answer either, because they treat acceptance by the application as validity [8], [9]; one of them notes that incomplete e-mail addresses may be submitted successfully because the application lacks the check [8], and a robustness study using status codes as its oracle cannot see failures that return success [16]. Where semantic validity is measured, a base eight-billion-parameter model produces domain-coherent values under a quarter of the time [17], with no non-LLM comparison.

This article measures that gap at the input boundary of one purpose-built application with an explicit rule reference, and compares LLM inputs against random, rule-based, boundary and reference inputs on the same outcome. We found no prior study that measures semantic validity at a web application's input boundary against an enumerated rule reference, compares LLM producers with non-LLM producers on that outcome, or reports what a semantic validation layer costs in the request path on commodity hardware. The study establishes five things.

- F1. Inputs that the application accepts break rules it does not enforce, and the effect is measurable without any model in the loop. Rules decidable by

Z. Rahimoon is an independent researcher (e-mail: zahidrahimoon22@gmail.com).

F. H. Khoso is with [Department, Institution, City, Country] (e-mail: [address]).

code alone were broken by 7.9 % of structurally valid model inputs and 10.3 % of random ones.

- F2. The gap belongs to the validation boundary, not to AI writers. Random structurally valid inputs broke rules more often than model inputs (54.5 % against 37.6 %), and the ordering holds in every sensitivity variant. Any producer that satisfies the schema without knowing the rules falls into the gap.
- F3. A judge model is not yet a usable oracle for this task, and its error is measurable. Calibrated against planted violations and known-valid inputs, the judge caught 78.6 % of violations while flagging 14.1 % of valid inputs, which moves the judgement-rule estimate from 22.4 % to 13.0 % once corrected. The calibration design itself transfers to any study that uses a model as ground truth.
- F4. No cheap validation layer closes the gap. Of six designs evaluated at a bounded false-positive rate on held-out fields, the best reached 25.3 % recall, most of what it caught were violations a line of code could have enforced, and training a classifier on the same embeddings did not help (1.9 %).
- F5. On commodity hardware the cost ordering is decisive. A rule layer adds at most 14 ms to p97.5 latency, an embedding layer 65 to 96 ms, and an LLM judge seconds per request, saturating on two of three endpoints at eight concurrent clients. The guardrail literature we reviewed reports no comparable CPU figures [18]–[21].

The testbed, harness, rule reference, raw data and analysis scripts are released, including the defects found during execution and how each was handled.

II. Background and Terminology

Structural validation is every check the application performs before business logic: the runtime schema (type, presence, length, pattern, range, enumeration, format and any refinements written into it), service-level checks and database constraints. An input passes when the application accepts it. Semantic validity is whether a structurally valid value is appropriate for the application's purpose, covering consistency across fields, domain plausibility, compliance with business rules and safety for downstream consumers such as an LLM component, operationalised as satisfying every rule in a frozen, enumerated reference. An input that passes structural validation and violates at least one rule is SV-SI (structurally valid, semantically invalid). The conditional SV-SI rate, the primary outcome, is the share of SV-SI inputs among the structurally valid inputs whose rules were all decided; without conditioning, a generator that produces more well-formed values would look worse merely by passing more often.

Terminology differs across communities. The OWASP Input Validation Cheat Sheet separates syntactic from semantic validation [22]; we count a range the developer wrote into the schema as structural, because the application enforces it. Security research speaks of logic or business logic vulnerabilities [4], [5], [23] and API testing of inter-parameter dependencies [24], [25]. In LLM security the adjective usually means adversarial intent [26], whereas our usage concerns whether the data is appropriate, which includes that case.

III. Related Work

Limits of structural validation. JSON Schema has a formal semantics and a known complexity [1], [2], and empirical studies describe which constraints practitioners encode [3]. Inter-parameter dependencies such as requires, only-one and arithmetic relations cannot be stated in an OpenAPI document [24], [25], and schema-derived fuzzers inherit the same boundary [27], and surveys of input-validation vulnerabilities remain organised around injection classes [28]. The rules beyond that boundary are the classical subject of logic-flaw research. Waler inferred likely invariants over session state and database query parameters and model-checked them [4], and related classes are detected by behavioural patterns [5], parameter tampering [6], mass-assignment testing [7] and excessive-data-exposure fuzzing [29]. Recent work applies LLMs to find such flaws in code [23]. In the studies we reviewed the inputs are crafted by an attacker or a tester, and they are not compared across producers.

LLM-generated inputs and schema-valid outputs. LLMs are better than random and type-based generators at producing inputs that applications accept: on thirty real forms an LLM-driven approach reaches a passing rate of 83 % against 23 % for a static type-based generator and 3 % for random crawling [9], and across 146 forms the best of eleven models submits over 98 % of its inputs [8]. The oracle in both cases is acceptance by the application. Where semantic validity is measured separately, a base model reaches 22.9 % domain-coherent values against 72.4 % after fine-tuning [17]; related work mines input constraints from specifications and benchmarks form-filling agents [10]. For model outputs, several 2026 studies document the same pattern, separating schema validity from semantic success and finding the second well behind [11]–[15], while tool-use benchmarks evaluate emitted calls against a task's expected answer [30], [31]. In all of them the semantic check is the oracle rather than a layer that is itself evaluated.

Data validation beyond schemas, and inappropriate input as a security problem. Data-management research validates data beyond structural schemas with anomaly detection for machine-learning pipelines [32], declarative constraint checks that act as unit tests for data [33] and learned detection of column semantic types [34], all on datasets in batch rather than on one request. Classifying a single field entry from a context embedding and statistics of recent entries appears in a patent [35], of which our EMB design is a simpler relative. On the security side, indirect prompt injection established that models do not separate data from instructions [36] and that attacks on deployed applications succeed [37]; prompt fragments entered through an ordinary form field can later drive a

database write [38], the pattern occurs in the wild [39], and benchmarks extend the coverage [40], [41]. Those inputs are attacker-crafted and mostly free text.

Detection layers and their cost. Input-side detectors include fine-tuned classifiers [18], embedding classifiers [19], [42], small transformers [43] and LLM judges [44], and what constrains deployment is the false-positive rate on benign traffic [45]. Cost evidence is thin and tied to GPUs: a detector presented as deployable gives no timing at all [18], an embedding guardrail puts its embedding floor at 35 ms on a datacentre GPU [19], and a layered framework reports a 61 ms median on an A100 with model inference excluded [20]. A systematisation notes that cost and utility are routinely overlooked [21], the one study comparing regex, classifier and LLM tiers head to head does not state its hardware [46], and LLM judges can themselves be redirected by the content they assess [47]. We found no measurement of a validator for data appropriateness on structured request payloads on CPU.

Position of this work. Three lines of work come closest, and this study differs from each in one specific way. The output-side studies measure values a model emits [11]–[13], [15], judged against a task's expected answer; we measure values an application accepts and stores, where nothing raises an error and the value is later read by people or by an assistant. The form and API testing studies count acceptance by the application [8], [9], so an inappropriate value that is accepted counts as a success, and the one study that measures domain coherence separately has no non-LLM comparison [17]. The guardrail studies evaluate detectors for adversarial intent and report cost, if at all, on datacentre GPUs [19], [20]; we evaluate detectors for ordinary data appropriateness on structured payloads on a four-core CPU, and report detection quality and cost together. The comparison across seven producers, including non-LLM ones, is what lets us ask whether the gap belongs to AI writers or to the validation boundary itself.

IV. Research Questions and Hypotheses

The following were frozen, with the primary metrics and the analysis plan, before any experimental input was generated. Prompt family P5 (injection, Section V) is excluded from all primary rates and analysed separately.

- RQ1. Do LLM-generated inputs pass structural validation while violating business rules? H1: the conditional SV-SI rate of the LLM conditions has a 95 % Wilson interval whose lower bound exceeds 0.05.
- RQ2. How does this vary across field categories? H2: rates differ across categories; free text and cross-field highest, simple scalar and structured (enumeration) lowest.
- RQ3. How do LLM inputs compare with other producers? H3a: LLM with field context exceeds random; H3b: LLM with field context exceeds benign-unusual reference inputs; H3c: LLM without field context exceeds random.
- RQ4. Which violation types dominate? Exploratory.
- RQ5. Can an added semantic layer detect SV-SI inputs? H5: at least one non-judge design reaches recall ≥ 0.70 at a false-positive rate (FPR) ≤ 0.05 on held-out fields.
- RQ6. What does it cost? H6: rule and embedding designs add at most 50 ms to p95 request latency while an LLM judge adds at least 1000 ms.

V. Method

Subject under study. The subject is a purpose-built Next.js 16 application written in TypeScript. It uses Zod 4 for validation on the client and the server, Prisma with SQLite for persistence, and a local Ollama runtime for two AI-facing features, support-ticket triage and a booking assistant. It has seven input surfaces and 39 fields. A reference file enumerates 43 business rules, of which five are enforced in code, two of those as Zod refinements (a birth date not in the future, a workshop end after its start); the remaining 38 are unenforced, a profile chosen to resemble small production applications. Each rule is classified in advance as objective, decidable from the input, the database and the clock (19 rules), or judgement (24 rules). The application and the rule reference were tagged before any input was generated (commit 62a7267), and the commit is recorded with every run. A purpose-built subject is a deliberate trade: real applications do not enumerate their business rules, and without a complete reference the semantic outcome has no ground truth.

Targets and generation conditions. Seventeen targets covering 22 fields represent every field category, every directly AI-facing field, and five cross-field pairs whose rule relates the two members; each carries a purpose description, its rule identifiers and its structural constraints. Seven conditions produce values for each target: A valid reference values; B random values that satisfy the schema, from a seeded generator of word sequences, numbers and dates within the schema's limits, with no attempt at meaning; C rule-based adversarial values, one template per unenforced rule; D an LLM given the field name and the structural constraints only; E the same LLM given the field's purpose and rules as well; F boundary values at the schema edges; and G benign but unusual reference values. Conditions A and G were specified as human-written. Their 272 values were first drafted with the help of a language model and checked against the schemas and the objective rules. The first author then went through every value, kept 262, replaced 3 and deleted 7. The application rejected all 3 replacements, which were promotional codes it does not recognise, so these conditions rest on the kept values and we call them human-curated rather than human-written.

LLM generation. Conditions D and E use qwen3:4b (Q4_K_M) through Ollama on a four-core CPU, at temperature 0.8, nucleus 0.9, with a fixed seed per run, three runs, and five values per call. Seven prompt families are stored as hashed files: P1 legitimate values, P2

unusual values, P3 values that satisfy the constraints but "would be wrong for the application to accept", P4 values whose meaning depends on downstream interpretation, P5 injection for AI-facing targets only, P6 boundary values, and P7 Unicode variants. Because P3 asks for violations, results are reported per family as well as pooled. The model identifier, digest, temperature, seed, prompt file and prompt hash are recorded for every value, and failed calls were counted and never regenerated: 88 failure records in 52 prompt cells, 75 of them call errors such as calls aborted at the 120 s timeout and 13 unparseable replies. A further 430 generated values used key names other than the target's; a harness defect had filled those keys from the form's defaults, so the records measure the defaults rather than the model and count as generation failures.

Structural measurement. Each input is measured twice. It is checked against the same schema object the application imports, and it is submitted over HTTP to the real route handler, which records the status, the service errors and the time. The structural outcome is the application's own decision, so an input rejected by one of the five enforced business checks counts as a structural rejection. Booking state is reset to the seed before each booking request.

Semantic ground truth. Objective rules are decided by rule functions. The 11 objective rules that apply to the 17 targets all have one. Judgement rules are decided by a judge model from a different family than the generator (gemma3:4b, temperature 0). The judge sees the field purpose, the rule text and the submitted value of the target's fields. It never sees the condition, the prompt family or the intent. Seven judgement rules depend on other fields of the same record, such as capacity being plausible for the location. Those were first judged without that context and were re-judged with the whole record; the re-judged verdicts are the ones reported, and a sensitivity analysis drops the seven rules altogether. A human annotator labelled 200 structurally valid inputs blind to condition, with 20 items repeated to estimate the annotator's consistency. Where a human verdict exists it takes precedence over the judge. An input is decided when every applicable rule has a verdict. For compute reasons the judge was applied to a stratified sample: 1200 structurally valid inputs drawn round-robin across target and condition with a fixed seed, plus 136 inputs for two booking targets that were re-measured after a defect (Section IX), up to 14 per stratum and as many as were available.

Analysis population and statistics. Of the 3881 inputs, the following are excluded: 430 key-mismatch records, 131 records produced by inadvertent retries of failed calls, 74 injection-family inputs, and 10 reference values that the reviewer deleted or replaced. That leaves 3236 inputs. Of these, 2917 passed structural validation and 1881 are decided. The remaining passes were either not in the judge sample (1001) or received no usable verdict (35). Rates carry Wilson 95 % confidence intervals (CIs) and contrasts are reported as odds ratios (ORs). H1 is evaluated on the lower bound of its interval, H2 with a χ^2 test and Cramér's V, and H3a to H3c with three one-sided Fisher tests under Holm correction. The judge sample gives the conditions different target mixes, so each contrast is also tested within target with the Cochran-Mantel-Haenszel (CMH) test, added after data collection. Values generated in one call, and targets within a category, are not independent, so Wilson intervals are too narrow; every main rate and contrast therefore also carries a cluster bootstrap interval (2000 draws over 502 generation calls and, separately, 17 targets). Every other analysis is labelled exploratory. It was fixed in advance that if judge-human agreement on judgement rules falls below $\kappa = 0.4$, every result depending on those rules is reported as exploratory. Every number here is generated from the raw data by released scripts. The hypotheses, metrics and analysis plan were fixed in a dated decision log before generation began and are released unchanged; version control over the analysis code started after data collection, so that log and the frozen subject tag are the evidence of pre-registration, not a commit history.

VI. Semantic Validation Layers

Six designs run as a drop-in check after structural validation. R packages the objective rule functions; because those also define the ground truth for objective rules, R cannot produce a false positive on them, so it is a floor rather than a competitor. EMB scores a value by its cosine distance from an embedding (nomic-embed-text) of the field's purpose and rules, using no labelled examples; EMB-TRAINED fits logistic regression on those same embeddings over the training folds, which is the design the frozen plan anticipated. SLM prompts a small language model (SLM; qwen3:1.7b) with the field purpose and rules for a verdict and a confidence. HYB runs R first and then EMB on what R passes. JUDGE sends the same prompt to qwen3:4b, the generator's model, as a reference point, so SLM and JUDGE receive a rubric similar to the ground-truth judge's. The prompted designs run at temperature 0, and a reply without a verdict counts as a pass, because the deployed layer fails open.

Evaluation is leave-fields-out over five folds. In each fold the threshold is the smallest score at which at most 5 % of the valid training-fold inputs are flagged, and that threshold is applied to the held-out fields. Designs are compared on recall at that operating point, with FPR, precision and the threshold-free area under the ROC curve (AUROC). The model-based designs were scored on a stratified sample of 573 decided inputs, of which 261 are SV-SI, and all designs are compared on that sample; the fast designs were also scored on all 1933 decided inputs, a population that includes the injection family and the retried records.

VII. Results

Rate of semantic violations (RQ1).

Among decided, structurally valid LLM inputs, 412 of 1096 (37.6 %, 95 % CI 34.8 to 40.5) broke at least

TABLE I
Structural and semantic outcomes by input-generation condition. The conditional SV-SI rate, the primary outcome, is the share of structurally valid inputs that violate at least one business rule, over decided passes only (every applicable rule received a verdict; passes outside the judge sample are excluded). Prompt family P5 (injection) and retried records are excluded; intervals are Wilson 95%. *Specified as human-written; values AI-drafted and then reviewed value by value by the researcher (human-curated).

Condition	n	Struct. pass (%)	Passes	Decided	SV-SI	Cond. SV-SI (%)	95% CI
A valid human*	134	97.8	131	130	17	13.1	[8.3, 19.9]
B random	425	93.6	398	365	199	54.5	[49.4, 59.6]
C rule-based adversarial	85	96.5	82	81	59	72.8	[62.3, 81.3]
D LLM context-free	1268	84.0	1065	532	238	44.7	[40.6, 49.0]
E LLM context-aware	1108	92.8	1028	564	174	30.9	[27.2, 34.8]
F boundary	85	96.5	82	81	28	34.6	[25.1, 45.4]
G benign-unusual human*	131	100.0	131	128	28	21.9	[15.6, 29.8]

one business rule (Table I), so H1 is supported. The pooled rate includes prompt family P3, which asks for rule violations; without P3 it is 35.9 %. By family the rate ranges from 28.8 % for legitimate-value prompts to 47.0 % when violations are requested, with context-dependent (32.0 %), unusual (38.4 %), Unicode-variation (40.4 %) and boundary (42.0 %) prompts in between. Every family is above the pre-registered bound. Agreement between the judge and the human annotator is below the pre-registered threshold ("How good is the ground truth?" below), so every rate depending on judgement rules is exploratory, this one included; the objective-rule rate depends only on the rule functions. The rate survives most sensitivity variants in Table II, including objective rules alone (7.9 %, CI 6.8 to 9.2). It does not survive the strictest one. One target dominates the objective violations: on workshop dates (F15+F16) nearly every model value was in the past, because the prompts never state the current date, a property of the harness rather than of the application. Removing both the judge and that target leaves 1.8 % (CI 1.3 to 2.5), whose lower bound is below the 0.05 threshold, so H1 is not supported on the one variant that depends neither on the judge nor on the date artefact. H1 is supported as pre-registered and not supported on judge-independent evidence alone. Per run the rate was 38.6, 50.3 and 45.9 % without field context and 33.3, 31.6 and 27.7 % with it; including the retried records gives 37.5 %. The reference conditions bound the labelling from both sides: 13.1 % of valid inputs were labelled invalid and 72.8 % of adversarial ones were caught.

Field categories (RQ2). Rates differ across categories, but H2 is counted as not supported: its directional prediction was that simple scalar and structured fields would be lowest, and structured came out highest. The association statistic ($\chi^2(6) = 213.0$, Cramér's $V = 0.34$) is reported descriptively only, because with one to four targets per category it tests heterogeneity between targets rather than between categories. These rates also rest on judgement rules, so they are exploratory. The pattern is as follows. Free text (51 %) and cross-field (45 %) are high, as expected, structured fields are the highest (95 %) rather than among the lowest, and financial fields are the lowest (25 %); AI-facing is 44 %, search 32 % and simple scalar 27 %. Categories contain between one and four targets, and per-category and per-target tables are in the repository. The structured category is a single target, a ticket's priority and category. The calibration below shows that the judge flagged 8 of 8 valid inputs on one of that target's rules (B-TK-4) even when given the whole record, so the reversal is a property of that rule's labelling rather than of enumerated fields in general.

TABLE II
Sensitivity of the conditional SV-SI rate to design choices. LLM: conditions D and E pooled; B: random; G: benign-unusual (human-curated). "Without context rules" drops the seven judgement rules that name other fields of the record. The two objective rows use the rule functions alone and do not depend on the judge. Target-weighted averages per-target rates by each target's share of structural passes. P1 and P3 apply to model conditions only.

Variant	LLM (%)	95% CI	B (%)	G (%)
Pre-registered (all rules)	37.6	[34.8, 40.5]	54.5	21.9
P1 (legitimate) prompts only	28.8	[23.1, 35.2]	n/a	n/a
Without P3 (asks for violations)	35.9	[32.9, 39.1]	54.5	21.9
Without dates (F15+F16)	29.1	[26.4, 32.1]	51.8	22.5
Objective rules only (no judge)	7.9	[6.8, 9.2]	10.3	0.8
Objective only, without dates	1.8	[1.3, 2.5]	5.1	0.0
Without context rules	24.5	[22.0, 27.1]	36.2	9.4
Target-weighted	38.9	n/a	54.5	22.1

TABLE III
Pre-declared one-sided contrasts on the conditional SV-SI rate (Fisher exact, Holm-corrected across the three tests). Each test is one-sided in the pre-registered direction (Rate 1 > Rate 2), so an odds ratio below 1 yields p close to 1. Conditions: B random, E LLM with field context, G benign-unusual (human-curated), D LLM without field context. Within-target results are in the text.

	Contrast	Rate 1 (%)	Rate 2 (%)	OR	p_{Holm}	Supported
H3a	E>B	30.9	54.5	0.37	1.00	no
H3b	E>G	30.9	21.9	1.59	0.08	no
H3c	D>B	44.7	54.5	0.68	1.00	no

Comparison of producers (RQ3).

None of the three pre-registered contrasts is supported, and two point the other way (Table III). They are computed on labels that include judgement rules, so under the agreement gate they are exploratory as well. The objective-rule sensitivity row shows the same direction without the judge. Random structurally valid inputs broke rules in 54.5 % of cases, more often than LLM inputs without field context (44.7 %) or with it (30.9 %), and that

holds in every sensitivity variant (Table II). Clustering changes how firm each contrast is. Resampling whole targets, the difference between context-aware model inputs and random inputs still excludes zero (-42.7 to -4.1), while the difference between context-free model inputs and random inputs does not (-24.7 to 5.2), and the pooled LLM rate has a target-clustered interval of 18.3 to 59.2, far wider than its Wilson interval. Within target (CMH, post hoc) the odds of a violation were lower for LLM inputs with context than for random inputs (odds ratio 0.18, 95 % CI 0.12 to 0.27) and lower without context (0.67, 0.46 to 0.97); adversarial inputs exceeded random ones (2.56, 1.39 to 4.72), and field context lowered the odds relative to the schema alone (0.22, 0.14 to 0.33). Condition E is given the very rules that define the outcome, so that reduction is an upper bound on what context can achieve rather than a deployable intervention: an application able to put its rules in a prompt can also enforce the objective ones in code. H3b is the one contrast that points the way it was predicted to, and it does not reach the threshold: LLM inputs with field context broke rules more often than benign-unusual reference inputs, but not detectably so when pooled ($p_{\text{Holm}} = 0.08$), only borderline within target (1.70, 0.97 to 2.97, $p = 0.05$), and resampling whole targets puts the difference at -8.3 to 32.5 percentage points, which includes zero. The reference inputs are also model-drafted values curated by a human reviewer, so this compares model input with human-curated input rather than with input a person wrote unaided.

An exploratory variation repeated conditions D and E on ten targets with families P1 to P4, changing only the generator. The 1.7-billion-parameter model produced more violations than the 4-billion model on objective rules (4.8 % against 1.1 %, OR 4.4, $p_{\text{Holm}} = 0.001$) and on all rules (35.0 % against 30.2 %, $p_{\text{Holm}} = 0.65$); neither the 8-billion model (23.1 %) nor temperature 0 (26.3 %) differed detectably. With one run per generator this only indicates that a smaller model may widen the gap.

Violation types (RQ4). Counts of judgement-rule violations depend on the judge and are exploratory. Judgement-rule violations outnumber objective ones (582 against 272, across all conditions). The single most frequent violation is objective: a workshop must start in the future (146 violations). The next two, the ticket priority rule (102) and the capacity rule (91), are the two rules the calibration below shows the judge cannot decide, so those counts measure judge behaviour as much as violation. A profile bio addressed to the assistant (64) and duration plausibility (57) follow. On AI-facing fields, 81 structurally valid LLM inputs violated a rule. Of those, 20 came from the injection family and 61 from the other prompt families.

How good is the ground truth? Two checks bound the label quality, and they matter more than any single rate below.

Calibration against the designed conditions. Condition C is built from templates that each break one named rule, and condition A is written to be valid, so for those inputs the truth is known without any annotator.

TABLE IV
Judge calibration on the two designed conditions, by judgement rule (rules with at least two designed violations or at least one false positive, ordered by false positives). Recall is measured on condition C, whose templates each break one named rule; the false-positive rate is measured on condition A, which is written to be valid. Full table in the repository.

Rule	Caught	Recall (%)	False pos.	FPR (%)
B-TK-4	2/2	100.0	8/8	100.0
B-CO-7	5/5	100.0	7/8	87.5
B-BK-7	5/5	100.0	2/8	25.0
B-PR-6	5/5	100.0	1/8	12.5
B-RV-3	1/2	50.0	1/8	12.5
B-CO-4	4/5	80.0	0/8	0.0
B-PR-5	5/5	100.0	0/7	0.0
B-CO-5	3/5	60.0	0/8	0.0

On judgement rules the judge caught 44 of 56 designed violations (recall 78.6 %, CI 66.2 to 87.3) and flagged 19 of 135 valid inputs (false-positive rate 14.1 %, CI 9.2 to 20.9). Per-rule figures are in Table IV. A false-positive rate of that size matters, because the observed judgement-rule rate among model inputs is only 22.4 %: correcting it for the measured recall and false-positive rate gives 13.0 %. Condition C is deliberately blatant, so its recall is optimistic and the corrected figure is a lower bound, but the gap between 22.4 % and 13.0 % attributes about 42.2 % of the judgement-rule signal in this study to judge error rather than to violation. The error is concentrated: 5 of the 17 calibrated rules produced any false positive at all, and the worst, B-TK-4, fired on 8 of 8 valid inputs even when the judge was given the whole record.

Agreement with a human. On judgement rules the judge agreed with the annotator on 70.6 % of 126 verdicts (Cohen's $\kappa = 0.24$), and the annotator was unsure on 3.6 %. Agreement is below the pre-registered 0.4, so every judgement-rule result in this article is exploratory. That annotation cannot settle which rater is at fault, because the same annotator also disagreed with the deterministic rule functions on objective rules (89.5 % agreement, $\kappa = 0.32$, 19 rule-function violations against 4), and those rules are decidable by code. A single annotator working at 3 s per item is therefore not an adequate reference, which is why the calibration above carries the argument instead. The annotator was self-consistent on repeated items ($\kappa = 0.71$).

Detection quality (RQ5).

No design reached the target, so H5 is not supported (Table V). Detection is measured against the same labels, so the judgement-rule part of these results is exploratory as well. The objective-rule part is not. At the operating point fixed on the training folds, the best non-judge design, HYB, detected 25.3 % of SV-SI inputs (CI 20.2 to 30.5) at a held-out FPR of 5.4 %, which is above the 5 % it was tuned for. Most of what it catches comes from R. HYB and R both caught 100.0 % of the inputs that violate only objective rules, and 6.2 % and 0.0 % of those that violate a judgement rule. On all decided inputs the fast designs reach 29.3 % for R and 32.5 % for HYB. EMB on its own

TABLE V
Detection quality of the candidate semantic validation layers on the shared stratified sample of 573 decided inputs (261 SV-SI, 312 valid), leave-fields-out (5 folds). Recall, 95% bootstrap interval, FPR and precision are at the pre-registered operating point: the threshold is the smallest score whose false-positive rate on valid training-fold inputs is at most 5%, applied to held-out fields. AUROC is threshold-free. The native column (exploratory) applies each design's own decision rule without tuning; JUDGE returned no usable verdict for 25% of inputs, counted as not flagged. R's zero FPR is partly by construction, because its rule functions also decide the ground truth for objective rules, and JUDGE shares a model family with the generator. Latency is the layer's own median decision time on the study CPU; HYB reuses embeddings cached by EMB, so its request-path cost is the one in Table VI. †EMB-TRAINED needs training folds, so it is scored on all decided inputs rather than on the shared sample.

Design	Recall (%)	95% CI	FPR (%)	Precision (%)	AUROC	Native recall / FPR (%)	Median (ms)
HYB	25.3	[20.2, 30.5]	5.4	79.5	0.59	22.6 / 1.6	0.01
R	20.3	[15.6, 25.2]	0.0	100.0	0.60	20.3 / 0.0	0.01
JUDGE	6.5	[3.7, 9.5]	1.0	85.0	0.73	45.2 / 8.7	2709
EMB	5.0	[2.5, 7.8]	5.4	43.3	0.45	2.3 / 1.6	31
SLM	1.9	[0.4, 3.7]	0.6	71.4	0.72	62.8 / 21.5	1075
EMB-TRAINED†	1.9	[1.1, 2.9]	3.5	27.3	0.43	n/a	n/a

was no better than chance (AUROC 0.45).

The prompted designs rank inputs better (AUROC 0.72 for SLM and 0.73 for JUDGE against 0.59 for HYB), but their scores take only a handful of values, so holding FPR to 5 % on the training fields forces the threshold to the top confidence level, where SLM detects 1.9 % and JUDGE 6.5 %. Taking each model's own verdict without tuning, which is exploratory, SLM detects 62.8 % while flagging 21.5 % of valid inputs, and JUDGE detects 45.2 % while flagging 8.7 % and gives no usable verdict for 25 % of inputs. Both receive a rubric similar to the ground-truth judge's, so on judgement rules their measured "recall" is largely agreement between small language models and the small model that produced the labels, not detection of violations, and R's recall on objective rules is tautological because the same rule functions decide the labels. The frozen plan anticipated a trained classifier, and the leave-fields-out labels support one: logistic regression on the same embeddings, fitted on the training folds, reaches 1.9 % recall (CI 1.1 to 2.9) at 3.5 % FPR over 1933 inputs, with AUROC 0.43. Training does not rescue the embedding design. Thresholds also did not transfer across fields: the held-out FPR per fold reached 26 % for the embedding-based designs.

Overhead (RQ6).

Each configuration ran against the production build with only its own model loaded, and no request or layer call failed in any run (Table VI). H6 is not supported, because its first part fails. R added at most 1 ms to p97.5 latency with one client and 14 ms with eight, within the 50 ms bound; the negative differences at eight clients reflect run-to-run variation. EMB added 65 to 96 ms with one client, above the bound, and 122 to 751 ms with eight, because every request embeds its text on the same CPU. On the p95 bracket used as a sensitivity check, EMB fails on the review endpoint and is undetermined on the other two. The second part of H6 holds. SLM added 527 to 534 ms at the median with one client and JUDGE added 1611 to 7704 ms. The model server answers one request at a time, so requests queue under concurrency. With eight clients SLM added 4149 to 4210 ms at the median, and JUDGE completed no request within the 60 s window in all three runs on the course endpoint and in two of three on the review endpoint. Throughput on the course endpoint fell from 47.4 requests per second with structural validation alone to 1.8 with SLM and 0.1 with JUDGE. Set against detection quality, only R and HYB lie on the cost-detection frontier: SLM, JUDGE and EMB each cost more and detect less than one of them at the pre-registered operating point. The frontier plot is released with the artefacts.

TABLE VI
Request-path overhead of each configuration (production build, four-core CPU, no GPU). Each value is the median over three 60 s runs, as the range across the three endpoints, relative to structural-only validation on the same endpoint and concurrency. H6 is evaluated on the added p97.5, the nearest tail percentile the load generator reports. "Ok" is the share of layer calls returning a verdict rather than failing open. ">60 000": saturated, no request completed in at least two of the three runs. Per-endpoint values are in the repository.

Config.	Δp50 (ms)	Δp97.5 (ms)	req/s	Ok (%)	H6
Concurrency 1					
structural	0	0	42.9 to 47.4	n/a	
R	0 to 2	0 to 1	41.9 to 46.0	100	holds
EMB	27 to 48	65 to 96	13.2 to 19.2	100	fails
HYB	26 to 45	65 to 82	13.6 to 19.2	100	
SLM	527 to 534	655 to 761	1.7 to 1.8	100	
JUDGE	1611 to 7704	7683 to 10824	0.1 to 0.5	100	holds
Concurrency 8					
structural	0	0	41.7 to 48.3	n/a	
R	2 to 6	-136 to 14	43.9 to 47.6	100	holds
EMB	126 to 288	122 to 751	16.0 to 27.9	100	fails
HYB	103 to 270	131 to 396	17.7 to 29.0	100	
SLM	4149 to 4210	6780 to 7180	1.7 to 1.7	100	
JUDGE	14282 to >60 000	23474 to >60 000	0.0 to 0.4	100	holds

VIII. Discussion

Throughout this section, every statement that rests on judgement rules is exploratory, because judge-human agreement fell below the pre-registered threshold. Statements about objective rules do not depend on the judge.

RQ1: the gap is measurable, and its size depends on what is asked. About a third of structurally valid LLM

inputs broke an unenforced rule, which extends output-side findings [11], [13], [15] to inputs an application would store. The magnitude is sensitive to prompt intent, to one date-dependent target, and to the judge, and the two checks it does not survive matter as much as the headline. The honest summary is that unenforced rules are broken often enough to matter, that this study does not establish the size of the effect, and that the ground truth is the limiting factor.

RQ2: where it occurs is uneven. Category is moderately associated with the outcome, but individual targets drive that association: one target with a clock-dependent rule accounts for over half of the cross-field violations, and the structured category is a single target on which the calibration shows the judge is unreliable.

RQ3: the gap is not specific to AI. This is the central negative result. Random values that satisfy the schema broke rules more often than model values, with or without field context, both pooled and within target, which points to the validation boundary rather than to the producer. A schema admits many values the application should reject, and any producer that satisfies the schema without knowing the rules will supply some of them. What distinguishes the model is that context helps: giving it the purpose and the rules cut the odds of a violation by 78 % relative to the schema alone. The random generator makes word sequences with no attempt at meaning, so it bounds the space rather than representing a person, and the failure of H3a and H3c carries little information by itself. The comparison that would settle the AI question, model input against input written unaided by several people, has not been run.

RQ4: most violations are ordinary. The single most frequent violation was of an objective rule the application could enforce directly, though judgement-rule violations remain the majority. On AI-facing fields most LLM violations came from ordinary prompts rather than injection attempts, so the gap is mostly unenforced business logic rather than attack.

RQ5: no cheap layer closes it. The fast designs that can be held to a low FPR, R and HYB, find about a quarter of the violations, and most of that is objective violations that could simply be enforced. JUDGE under the same constraint finds 6.5 %, again mostly objective. The prompted models rank inputs better than the rest (AUROC 0.73 and 0.72) but produce coarse, field-dependent scores, so the constraint pushes them to an all-or-nothing threshold; left at their own decision rule they catch far more while rejecting 8.7 % and 21.5 % of valid inputs. Detecting judgement-rule violations remains the open problem.

RQ6: cost rises faster than detection. On commodity CPU hardware only the rule layer is effectively free, and it detects violations of rules the application could have enforced in the first place. The embedding layer costs tens of milliseconds and detected almost nothing. The prompted models cost half a second to several seconds per request with one client, a local model server handles one request at a time so they cannot keep up with eight, and their better ranking of inputs does not survive the false-positive constraint. Two caveats bound that. Run-to-run variation at eight clients reaches the same order as the embedding effect that fails H6 at one client, so the embedding verdict is marginal; and the serialisation belongs to one local model server, since a hosted judge would cost roughly a second per call and scale horizontally. On this hardware the practical recommendation is to enforce objective rules in code and to treat judgement rules as an open problem, rather than to add a model to the request path.

Alternative explanations. The six explanations fixed in advance fare as follows. Weak business rules rather than AI: consistent with the data, because the gap appears for every producer that does not know the rules. Random inputs produce the same effect: consistent, and random inputs exceed model inputs. Unusual human inputs behave the same: not resolved. Context-aware model inputs broke rules somewhat more often than human-curated unusual ones (1.70, 0.97 to 2.97), but the pooled test does not reach the threshold and the target-clustered difference includes zero. Confined to free text: not supported, because structured and cross-field targets also show high rates, although single targets drive them. Prompt injection under another name: not supported. Injection prompts are excluded from all rates and account for 20 of 81 LLM violations on AI-facing fields. Too expensive to deploy: supported for the model-based layers on CPU hardware, and not supported for rules.

IX. Threats to Validity

Internal validity. Labelling is the main threat. Judgement rules were decided by one 4B judge (agreement with a human annotator $\kappa = 0.24$ on 126 shared verdicts, below the pre-registered 0.4, so judgement-rule results are exploratory). The calibration shows the re-judge did not make the seven cross-field rules reliable, so the sensitivity row that drops all seven is still reported: it lowers the LLM rate to 24.5 % and the random rate by more (Table II), leaving the RQ3 direction intact. Agreement on semantic judgements can be low even between humans [48]. The disagreement here is not one-sided noise. On the shared verdicts the judge called 37 violations against the annotator's 28, so the judge is the stricter rater and judgement-rule rates may be too high, and it returned no usable verdict on 36 further rules the annotator decided. The annotator in turn disagreed with the deterministic rule functions on objective rules ($\kappa = 0.32$), which decide such rules from the record and the clock, so part of the low agreement is annotation error or the limited context the annotator saw rather than judge error alone. The judge sample gives the conditions different target mixes, and target-weighted rates are close to the pooled ones. Time-relative rules are evaluated against the clock at the moment the labels are merged, so a later re-run can change labels near the boundary, and the judge ran once, its run-to-run stability on this CPU unmeasured.

Defects found and corrected. Eight defects were found during execution, each recorded as a deviation with the invalid data preserved: the database was not reset between booking measurements, so capacity ran out and those targets were re-measured; a resume routine retried failed generation calls, whose records are excluded; model outputs with other key names were silently replaced by form defaults, and those records are excluded as generation failures; a first overhead run never enabled the layer and is discarded; the first detector analysis chose thresholds with a quantile, which fails for coarse scores; the label merge never passed human verdicts to the ground-truth algorithm while the importer dropped cells after an empty one; the judge was asked to decide seven cross-field rules without the other fields, so those rules were re-judged with the whole record; and the merge appended rather than rewrote its output, so re-running it counted every input twice. Each was fixed before any figure computed from it was reported. Three were found by separate adversarial review passes over the code and the analysis outputs, and one by an academic reviewer reading the draft.

External validity. The study covers one application in one domain, built by the first author, with one enforcement profile, so the results may not transfer. Generation uses quantised open models of 1.7 to 8 billion parameters on CPU; frontier models may behave differently, and evidence that scale does not predict semantic quality [13], [15] cuts both ways. The prompts never state the current date, which inflates date-rule violations for model inputs.

Construct validity. Semantic invalidity is defined by a frozen reference another team would write differently. It was frozen before any input existed and violations are reported per rule, so a reader can discount rules they disagree with.

Conclusion validity. Pooling three runs inflates the LLM sample, and per-run rates are reported for that reason. The category mapping assigns dual-tagged fields to one category, which was decided before the data existed but not logged at the time. The three contrasts H3a to H3c were the only pre-declared ones, but they are computed on labels that include judgement rules, so they too are exploratory under the pre-registered agreement gate, and only results decided by the rule functions alone remain confirmatory. H3b in particular compares model input with human-curated input, not with input written unaided, because the reference conditions were model-drafted before the first author reviewed them.

Who produced the human data. The first author carried out both the review and the annotation, and also built the subject application and wrote the rule reference. The annotation is blind to the generation condition, because the interface never shows it, but it is not independent of the study, and one annotator cannot establish how far the disagreement with the judge generalises. The review took 19 minutes for 272 values and the annotation 33 minutes for 200 items, a median of 3 s per item, both well below the effort budgeted, which may itself contribute to the low agreement. The per-item answers and their timestamps are released.

X. Conclusion and Future Work

Schema validation answers whether an input is well formed, not whether it belongs. At the input boundary of one application with an explicit rule reference, structurally valid inputs broke unenforced business rules often enough to matter, and random inputs broke them more often than model inputs, so the gap belongs to the boundary rather than to AI writers. How large the gap is remains open: the judge-independent estimate falls below the pre-registered bound once a prompt artefact is removed, and correcting for the judge's false-positive rate removes about 42.2 % of the judgement-rule signal. The single most frequent violation was of an objective rule a line of validation code could enforce, and none of the candidate layers, trained or untrained, detected the pre-registered share of violations at a tolerable false-positive rate. A human annotation of 200 inputs agreed with the judge only at $\kappa = 0.24$, below the threshold fixed in advance, so the ground truth for judgement rules is the weakest part of the study. On a four-core CPU, rules added at most 14 ms at p97.5, while at the median a small language model added about half a second and an LLM judge several seconds, saturating with eight concurrent clients.

Future work should strengthen the ground truth with several annotators and more time per item, replace the rules the calibration shows the judge cannot decide, collect reference inputs written unaided by several people, and repeat the protocol on an existing open-source application.

Acknowledgment

The testbed, the harness, the rule reference, every generated input, measurement and label, and the scripts that produce every table, figure and number in this article are available at https://github.com/zahidrahimoon/semantic-validation-gap. Generative AI tools assisted with drafting and code; the authors verified all content and are responsible for it.

References

[1] F. Pezoa, J. L. Reutter, F. Suarez, M. Ugarte, and D. Vrgoč, "Foundations of JSON schema," in Proceedings of the 25th International Conference on World Wide Web, ser. WWW '16. International World Wide Web Conferences Steering Committee, Apr. 2016, pp. 263–273.

[2] L. Attouche, M.-A. Baazizi, D. Colazzo, G. Ghelli, C. Sartiani, and S. Scherzinger, "Validation of modern JSON schema: Formalization and complexity," Proceedings of the ACM on Programming Languages, vol. 8, no. POPL, pp. 1451–1481, 2024.

[3] M.-A. Baazizi, D. Colazzo, G. Ghelli, C. Sartiani, and S. Scherzinger, An Empirical Study on the "Usage of Not" in Real-World JSON Schema Documents. Springer International Publishing, 2021, pp. 102–112.

[4] V. Felmetsger, L. Cavedon, C. Kruegel, and G. Vigna, "Toward automated detection of logic vulnerabilities in web applications," in Proceedings of the 19th USENIX Security Symposium. USENIX Association, 2010. [Online]. Available: https://www.usenix.org/legacy/event/sec10/tech/full_papers/Felmetsger.pdf

[5] G. Pellegrino and D. Balzarotti, "Toward black-box detection of logic flaws in web applications," in Proceedings 2014 Network and Distributed System Security Symposium, ser. NDSS 2014. Internet Society, 2014.

[6] P. Bisht, T. Hinrichs, N. Skrupsky, R. Bobrowicz, and V. N. Venkatakrishnan, "NoTamper: automatic blackbox detection of parameter tampering opportunities in web applications," in Proceedings of the 17th ACM conference on Computer and communications security, ser. CCS '10. ACM, Oct. 2010, pp. 607–618.

[7] D. Corradini, M. Pasqua, and M. Ceccato, "Automated black-box testing of mass assignment vulnerabilities in RESTful APIs," in 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, May 2023, pp. 2553–2564.

[8] T. Li, C. Cui, R. Huang, D. Towey, and L. Ma, "Large language models for automated web-form-test generation: An empirical study," ACM Transactions on Software Engineering and Methodology, vol. 35, no. 3, pp. 1–37, Feb. 2026.

[9] P. Alian, N. Nashid, M. Shahbandeh, and A. Mesbah, "Semantic constraint inference for web form test generation," in Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ser. ISSTA '24. ACM, Sep. 2024, pp. 932–944, arXiv:2402.00950.

[10] B. Li, Y. Wang, H. Fei, J. Li, W. Ji, M.-L. Lee, and W. Hsu, "FormFactory: An interactive benchmarking suite for multimodal form-filling agents," in Proceedings of the 33rd ACM International Conference on Multimedia. ACM, 2025, pp. 13 273–13 280.

[11] Y. Li, "When JSON is not enough: Semantic reliability of schema-constrained LLM ordering agents," arXiv preprint arXiv:2607.18261, 2026.

[12] J. Ray, "The constraint tax: Measuring validity-correctness tradeoffs in structured outputs for small language models," arXiv preprint arXiv:2605.26128, 2026.

[13] A. K. Singh, H. V. Khurdula, Y. D. Khemlani, and V. Agarwal, "The structured output benchmark: A multi-source benchmark for evaluating structured output quality in large language models," arXiv preprint arXiv:2604.25359, 2026.

[14] A. Sigdel and R. Baral, "Schema first tool APIs for LLM agents: A controlled study of tool misuse, recovery, and budgeted performance," arXiv preprint arXiv:2603.13404, 2026.

[15] X. Wrenn, R. Raykov, A. Angelov, H. Kitahara, Y. Watanabe, and A. Sailer, "Evaluating LLM trade-offs for enterprise automation: Lessons from workflow generation in a production enterprise platform," arXiv preprint arXiv:2608.03311, 2026.

[16] H. G. Tigulla and M. Vieira, "LLM-based robustness testing of microservice applications: An empirical study," arXiv preprint arXiv:2605.14202, 2026.

[17] M. Kim, S. Sinha, and A. Orso, "LlamaRestTest: Effective REST API testing with small language models," Proceedings of the ACM on Software Engineering, vol. 2, no. FSE, pp. 465–488, Jun. 2025.

[18] D. Jacob, H. Alzahrani, Z. Hu, B. Alomair, and D. Wagner, "PromptShield: Deployable detection for prompt injection attacks," in Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy, ser. CODASPY '25. ACM, Jun. 2025, pp. 341–352, arXiv:2501.15145.

[19] I. Ahmed, A. A. Borsha, R. D. Prangon, A. fuad Ahmad, and T. H. Tran, "Reflex-guard: A low-latency guardrail for LLM prompt safety using dense semantic embeddings," arXiv preprint arXiv:2608.17556, 2026.

[20] G. Saleem, N. Ahmed, M. I. Zaman, A. Hassan, and U. Mujahid, "A layered security framework against prompt injection in RAG-based chatbots," arXiv preprint arXiv:2606.19660, 2026.

[21] X. Wang, Z. Ji, W. Wang, Z. Li, D. Wu, and S. Wang, "SoK: Evaluating jailbreak guardrails for large language models," in 2026 IEEE Symposium on Security and Privacy (SP). IEEE, 2026, pp. 39–58.

[22] OWASP Cheat Sheet Series Team, "Input validation cheat sheet," OWASP Cheat Sheet Series, OWASP Foundation, 2026, accessed: Sep; 22, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

[23] B. Metin, M. Wynn, A. Tunalı, and Y. Kepir, "Business logic vulnerabilities in the digital era: A detection framework using artificial intelligence," Information, vol. 16, no. 7, p. 585, Jul. 2025.

[24] A. Martin-Lopez, S. Segura, C. Muller, and A. Ruiz-Cortes, "Specification and automated analysis of inter-parameter dependencies in web APIs," IEEE Transactions on Services Computing, vol. 15, no. 4, pp. 2342–2355, Jul. 2022.

[25] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés, A Catalogue of Inter-parameter Dependencies in RESTful Web APIs. Springer International Publishing, 2019, pp. 399–414.

[26] N. Singh, A. Abuadbba, Y. Gao, S. Nepal, and H. Kim, "Shifting from injection to interaction: Rethinking web security in the age of LLMs and beyond," arXiv preprint arXiv:2609.03999, 2026.

[27] Z. Hatfield-Dodds and D. Dygalo, "Deriving semantics-aware fuzzers from web API schemas," arXiv preprint arXiv:2112.10328, 2021, a two-page tool demonstration appeared at ICSE '22 Companion, doi 10.1145/3510454.3528637; this nine-page version is the one cited.

[28] F. Faisal Fadlalla and H. T. Elshoush, "Input validation vulnerabilities in web applications: Systematic review, classification, and analysis of the current state-of-the-art," IEEE Access, vol. 11, pp. 40 128–40 161, 2023.

[29] L. Pan, S. Cohney, T. Murray, and V.-T. Pham, "EDEFuzz: A web API fuzzer for excessive data exposures," in Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ser. ICSE '24. ACM, Feb. 2024, pp. 1–12.

[30] P. Yu, W. Liu, Y. Yang, J. Li, Z. Zhang, X. Feng, and F. Zhang, "Benchmarking LLM tool-use in the wild," in Proc. Int. Conf. Learning Representations (ICLR), 2026, arXiv:2604.06185. [Online]. Available: https://openreview.net/forum?id=yz7fL5vfpn

[31] G. Yu, X. Sun, Z. Huang, S. Fan, Z. Zhang, X. Hu, X. Xue, Y. Shi, X. Xiao, Y. Song, B. Lyu, R. Wen, X. Li, Q. He, S. Zhu, and Z. Liu, "Getting the parameters right: A difficulty-graded benchmark and probe-guided training for LLM tool calls," arXiv preprint arXiv:2608.03071, 2026.

[32] N. Polyzotis, M. Zinkevich, S. Roy, E. Breck, and S. E. Whang, "Data validation for machine learning," in Proc. Machine Learning and Systems (MLSys), vol. 1, 2019. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2019/hash/928f1160e52192e3e0017fb63ab65391-Abstract.html

[33] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," Proc. VLDB Endowment, vol. 11, no. 12, pp. 1781–1794, 2018.

[34] M. Hulsebos, K. Hu, M. Bakker, E. Zgraggen, A. Satyanarayan, T. Kraska, Ç. Demiralp, and C. Hidalgo, "Sherlock: A deep learning approach to semantic data type detection," in Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining, 2019, pp. 1500–1508.

[35] A. Banerjee and V. T. Subrahmaniam, "Deep learning based context embedding approach for detecting data entry errors," US Patent 12 288 159, Apr. 29, 2025. [Online]. Available: https://patents.google.com/patent/US12288159B2

[36] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, ser. CCS '23. ACM, Nov. 2023, pp. 79–90, arXiv:2302.12173.

[37] Y. Kaya, A. Landerer, S. Pletinckx, M. Zimmermann, C. Kruegel, and G. Vigna, "When AI meets the web: Prompt injection risks in third-party AI chatbot plugins," in 2026 IEEE Symposium on Security and Privacy (SP). IEEE, 2026, pp. 4223–4242.

[38] R. Pedro, M. E. Coimbra, D. Castro, P. Carreira, and N. Santos, "Prompt-to-SQL injections in LLM-integrated web applications: Risks and defenses," in 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). IEEE, Apr. 2025, pp. 1768–1780, arXiv:2308.01990.

[39] S. Khodayari, X. Zhang, B. Acharya, and G. Pellegrino, "Indirect prompt injection in the wild: An empirical study of prevalence, techniques, and objectives," arXiv preprint arXiv:2604.27202, 2026.

[40] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1, ser. KDD '25. ACM, Jul. 2025, pp. 1809–1820.

[41] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, and K. Chaudhuri, "WASP: Benchmarking web agent security against prompt injection attacks," in Advances in Neural Information Processing Systems 38. Neural Information Processing Systems Foundation, Inc., 2025, pp. 22 436–22 458.
[42] M. A. Ayub and S. Majumdar, "Embedding-based classifiers can detect prompt injection attacks," in Proceedings of the Conference on Applied Machine Learning in Information Security (CAMLIS 2024), ser. CEUR Workshop Proceedings, R. Allen, S. Samtani, E. Raff, and E. Rudd, Eds., vol. 3920. Arlington, Virginia, USA: CEUR-WS.org, 2024, pp. 257–268. [Online]. Available: https://ceur-ws.org/Vol-3920/paper15.pdf
[43] Y. Datta and S. Rajasekar, "JavelinGuard: Low-cost transformer architectures for LLM security," arXiv preprint arXiv:2506.07330, 2025.
[44] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Y. Feng and E. Lefever, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 431–445.
[45] H. Li and X. Liu, "InjecGuard: Benchmarking and mitigating over-defense in prompt injection guardrail models," arXiv preprint arXiv:2410.22770, 2024.
[46] A. C. Maiorano, "Evaluating prompt injection defenses for educational LLM tutors: Security-usability-latency trade-offs," arXiv preprint arXiv:2605.06669, 2026.
[47] J. Shi, Z. Yuan, Y. Liu, Y. Huang, P. Zhou, L. Sun, and N. Z. Gong, "Optimization-based prompt injection attack to LLM-as-a-judge," in Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, ser. CCS '24. ACM, 2024, pp. 660–674.
[48] T. Calò, A.-E. Gurita, and L. De Russis, "Measuring the semantic accessibility gap in LLM-generated web UIs," in Proceedings of the Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems, ser. CHI EA '26. ACM, Apr. 2026, pp. 1–5.